

Presentación del sistema - Entregable D4

BÓVEDA SEGURA

EQUIPO 5

Hernández Ramírez Miguel Angel

Hernández Gutiérrez Carlos Mario

Rivera Lopez David Zaid

Solís Espinosa Andrea Vianney

Suárez Román Clara Alin



Índice



1. Descripción general del sistema

2. Arquitectura

3. Modelo de amenazas

4. Diseño criptográfico

5. Demostración

Descripción general del sistema



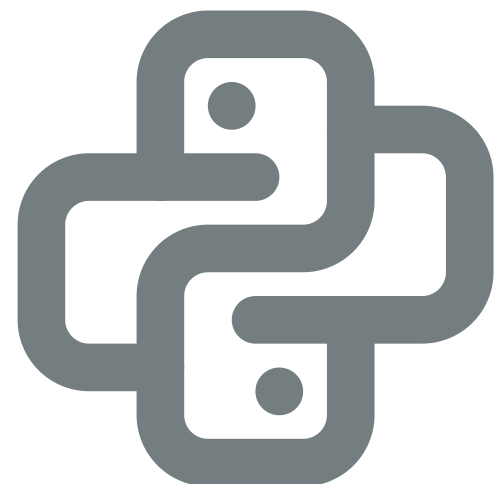
¿Qué problema resolvemos?

Compartir un archivo por canales no confiables como nube, USB, red no garantiza confidencialidad ni integridad del contenido.

Solución

- **Confidencialidad:** Solo los destinatarios autorizados pueden leer el archivo
- **Integridad:** Cualquier modificación al archivo es detectada automáticamente
- **Control de acceso:** El remitente decide exactamente quién puede descifrar





¿Qué hace el sistema?



Cifrado de archivos

AES-GCM-256
Cualquier tipo de archivo

Compartición segura

Múltiples destinatarios
RSA-OAEP por usuario

Gestión de claves

Par RSA por usuario
Identidad criptográfica

Integridad verificada

Auth tag 128 bits
Tamper detection

Firma digital

Autenticidad del remitente
No repudio

CLI funcional

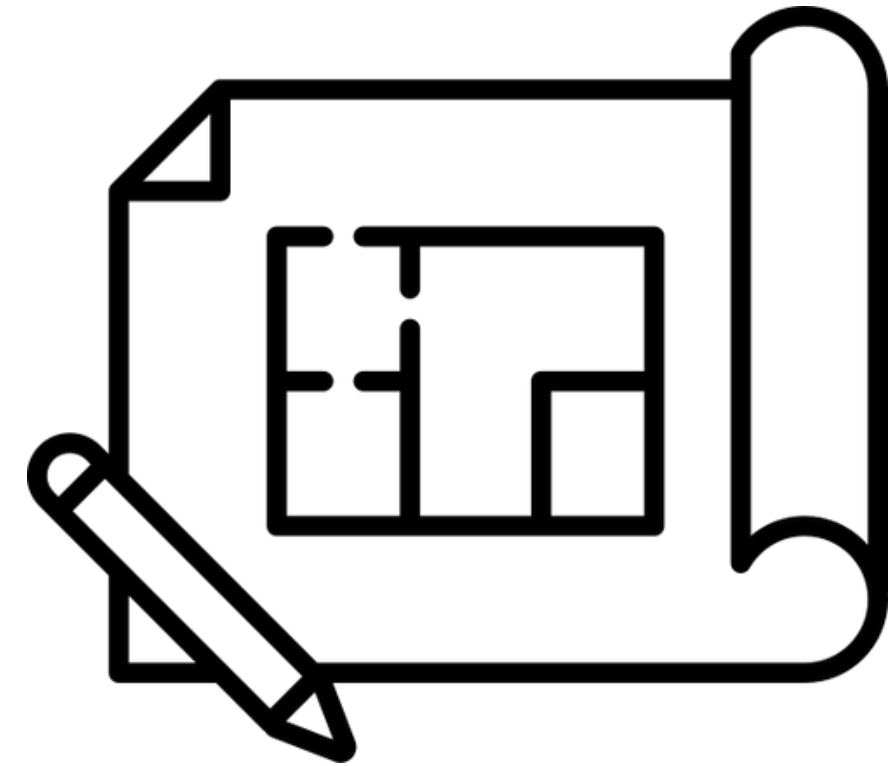
identidad / cifrar
descifrar

Stack tecnológico

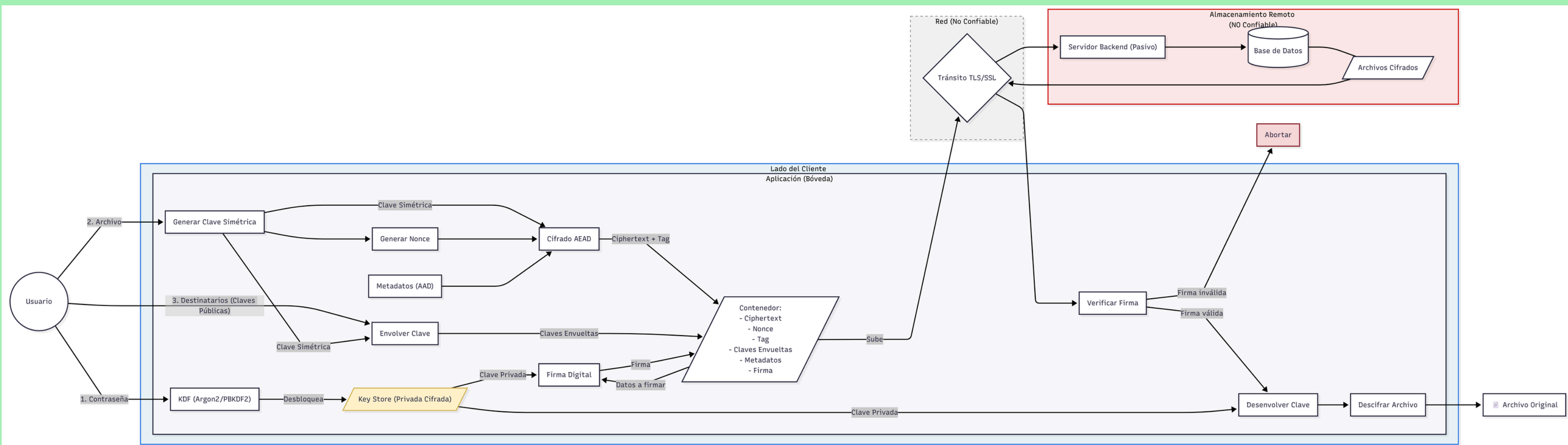
Python 3.10+ · PyCA cryptography · pytest · CLI con argparse

Arquitectura del Sistema

- Arquitectura Híbrida
- **Cifrado Simétrico:** AES-GCM-256.
- **Protección de Claves por cifrado asimétrico:** RSA-OAEP con SHA-256.
- **Contenedor Final:** Formato .vault.



Arquitectura del Sistema



Contenedor .vault



```
{
  "header": {
    "filename": "documento.txt",
    "recipients": ["a1b2c3..."],
    "version": "2.0"
  },
  "recipients": [
    { "recipient_id": "a1b2c3...",
      "encrypted_key": "3f8a7b..." }
  ],
  "nonce": "a1b2c3d4e5f6...",
  "ciphertext": "7f3a9c...e8b201"
}
```

- **Header (AAD):** metadatos autenticados
- **Recipients:** claves cifradas por usuario
- **Nonce :** valor único del cifrado
- **Ciphertext:** contenido protegido + auth tag

Modelo de amenazas



¿Contra quién nos defendemos?

- A1 — **Atacante externo:** Tiene todos los .vault. No tiene claves y no puede leer nada.
- A2 — **Destinatario malicioso:** Puede descifrar lo suyo. No puede suplantar al remitente ni modificar el archivo sin ser detectado.
- A3 — **Man-in-the-Middle:** Intercepta el contenedor. Cualquier byte modificado provoca un error de autenticación



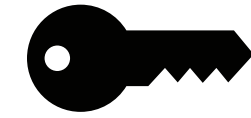
**RS-1 Confidencialidad
del contenido**

AES-256



**RS-3 Autenticidad del
remitente**

Firmas digitales



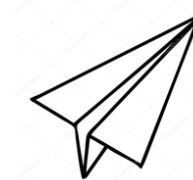
**RS-7 Separación de
claves**

Nueva clave por
archivo



RS-2 Integridad

Auth tag 128 bits



RS-5 Anti-tampering

Lista de destinatarios
en el AAD

Requisitos de seguridad y supuestos

Diseño criptográfico

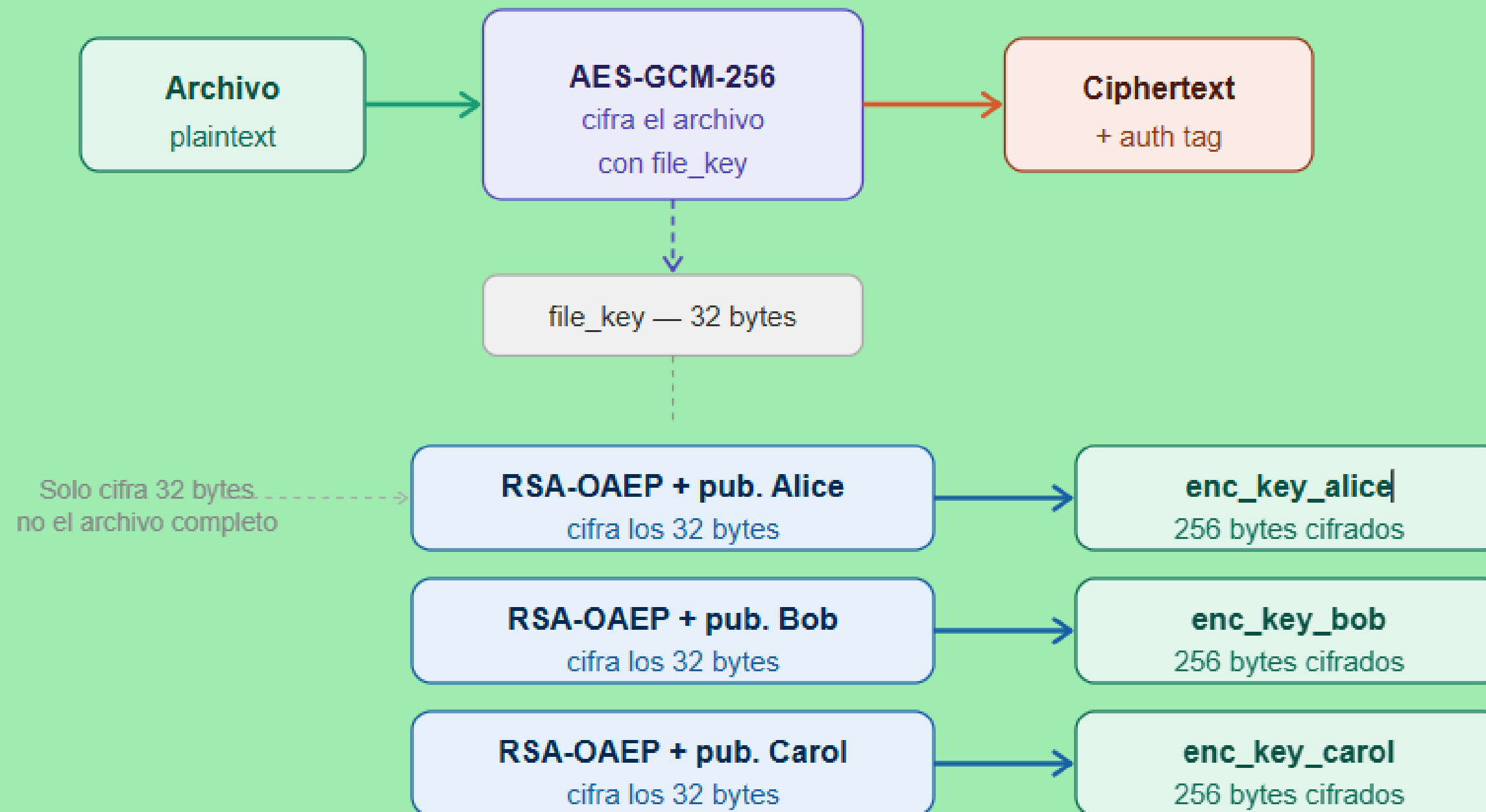


D2 — AES-GCM-256: parámetros criptográficos

Parámetro	Valor	¿Por qué?
Algoritmo	AES-GCM	AEAD: confidencialidad + integridad en una sola operación
Clave	256 bits	Nueva por archivo tenemos 2^{256} posibles claves
Nonce	96 bits	Aleatorio por cifrado nunca se repite
Auth tag	128 bits	Detecta cualquier modificación · 1 en 2^{128} de falsificar

Diseño criptográfico

D3 — Cifrado híbrido: AES-GCM + RSA-OAEP



Identificación de destinatarios: fingerprint



Clave pública

formato DER bytes canónicos



SHA-256 de esos bytes

Hay un seguimiento claro del dinero



Primeros 32 caracteres hexadecimales

ID único del destinatario

```
def get_key_fingerprint(public_key) -> str:
    pub_bytes = public_key.public_bytes(
        encoding=serialization.Encoding.DER,
        format=serialization.PublicFormat.SubjectPublicKeyInfo,
    )
    return hashlib.sha256(pub_bytes).hexdigest()[ :32]
```

Identificación de destinatarios: fingerprint



Clave pública

formato DER bytes canónicos



SHA-256 de esos bytes

Hay un seguimiento claro del dinero

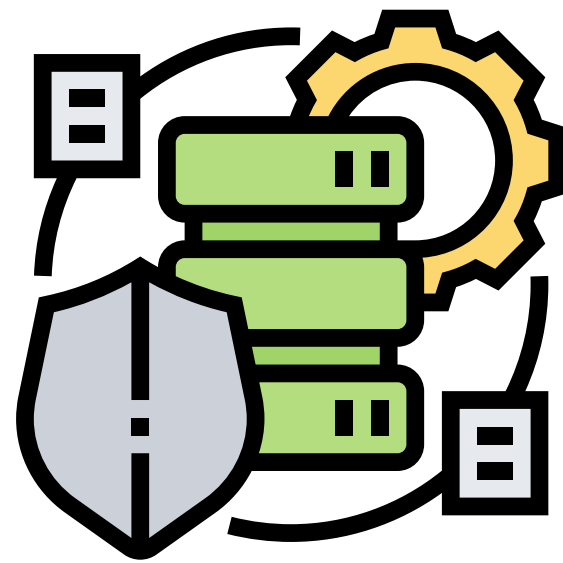
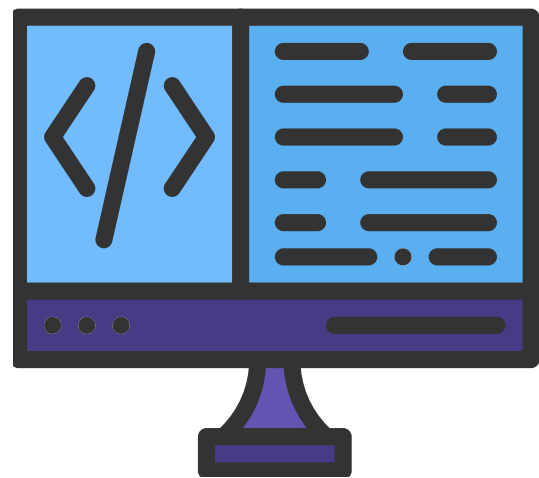


Primeros 32 caracteres hexadecimales

ID único del destinatario

Proteccion de metadatos

AAD



Datos Asociados Autenticados en el motor AES-GCM.

- Nombre del archivo, versión y lista de destinatarios autorizados.
- Defensa contra Adversario A3 (Man-in-the-Middle): Evita la adición de usuarios falsos.
- Evita la eliminación de usuarios legítimos.
- Evita el intercambio de identidades.

Resultado (Fallo Seguro): Cualquier alteración en el JSON rompe el Auth Tag de 128 bits y aborta el descifrado instantáneamente.



DEMO

4 escenarios:

- **Cifrado y descifrado válido:** Alice descifra
- **Archivo compartido:** otro destinatario puede descifrar
- **No-destinatario no puede descifrar:** intruso falla
- **Archivo modificado:** descifrado falla.

Link del video de la demostración:

[Link Video](https://drive.google.com/drive/folders/177D_PaRaNLxdHd_89z_RbrPCy9FQpU4X?usp=drive_link)

https://drive.google.com/drive/folders/177D_PaRaNLxdHd_89z_RbrPCy9FQpU4X?usp=drive_link